OS Orange Problem-2

Name: M LOUIS ALAN

Section:AIML H

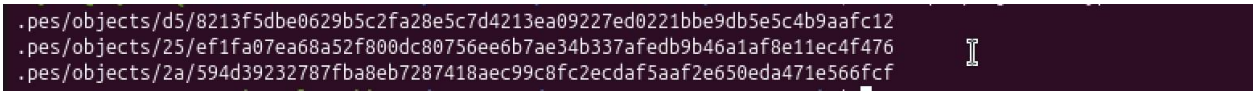
SRN:PES1UG24AM437

Phase 1: Object Storage Foundation

📸 **Screenshot 1A:** Output of ./test_objects showing all tests passing.



📸 **Screenshot 1B:** find .pes/objects -type f showing the sharded directory structure.



Phase 2: Tree Objects

📸 **Screenshot 2A:** Output of ./test_tree showing all tests passing.



Phase 3: The Index (Staging Area)

📸 **Screenshot 3A:** Run ./pes init, ./pes add file1.txt file2.txt, ./pes status — show the output.

```
echo "hello" > file.txt
./pes add file.txt
./pes status
Initialized empty PES repository in .pes/
Staged changes:
  staged:     file.txt
```

Screenshot 3B: cat .pes/index showing the text-format index with your entries.

```
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776263502 6 file.txt
```

Phase 4: Commits and History

Screenshot 4A: Output of ./pes log showing three commits with hashes, authors, timestamps, and me

```
commit c3d93555ee835801e44c819240fa91266985e1d50658165867339e76a9e43c73
Author: PES User <pes@localhost>
Date:   1776264153

    Commit 33

Commi

commit c9a239536c2b9429a585bd113987faef5c08fb08a01124b77749bb7836e0a124
Author: PES User <pes@localhost>
Date:   1776264153

    Commit 23

Commi

commit 1d6ca6ac3403dec060801c075e8c1faf66764523c378aae7a562fda5d5ca1641
Author: PES User <pes@localhost>
Date:   1776264153

    Commit 1
```

Screenshot 4B: find .pes -type f | sort showing object store growth after three commits.

```
.pes/HEAD
.pes/index
.pes/objects/1d/6ca6ac3403dec060801c075e8c1faf66764523c378aae7a562fda5d5ca1641
.pes/objects/71/4a8ac694436722923eb810c0211850ce4b6138927afed3f3a92450f585fe42
.pes/objects/a9/3994064c9f0621147b612bdf841da5485745e7024e25ea00e8e38202eb4b30
.pes/objects/b4/ee4550b0b7ad2965e05927480da3474a5b0286971433c4bb24213dcf015713
.pes/objects/b9/ed2198ca3f2ef819afcc21c7f8fef5e60689c2ddfa9bbe6f78f1d88f803edb
.pes/objects/c3/d93555ee835801e44c819240fa91266985e1d50658165867339e76a9e43c73
.pes/objects/c5/55a7730d53c5a8f89e29178f10c81b6202d9dfe23e67dd2331ef963fd3b90c
.pes/objects/c9/a239536c2b9429a585bd113987faef5c08fb08a01124b77749bb7836e0a124
.pes/objects/d4/78b38ba1414b841e3b6c393fd2d355d4fdfc5a38d40a45632113104bc7e874
.pes/refs/heads/main
```

Screenshot 4C: cat .pes/refs/heads/main and cat .pes/HEAD showing the reference chain.

```
c3d93555ee835801e44c819240fa91266985e1d50658165867339e76a9e43c73
```

Phase 5 & 6: Analysis-Only Questions

Branching and Checkout

Q5.1: A branch in Git is just a file in .git/refs/heads/ containing a commit hash. Creating a branch is creating a file. Given this, how would you implement pes checkout <branch> — what files need to change in .pes/, and what must happen to the working directory? What makes this operation complex?

Ans: pes checkout <branch> should update .pes/HEAD to point to the target branch ref, and the branch's ref file in .pes/refs/heads/ stays unchanged. The working directory must be rewritten to match the target branch tree, which is complex because it needs file-by-file updates, deletions, and conflict checks.

Q5.2: When switching branches, the working directory must be updated to match the target branch's tree. If the user has uncommitted changes to a tracked file, and that file differs between branches, checkout must refuse. Describe how you would detect this "dirty working directory" conflict using only the index and the object store.

Ans:

Check the index against the object store and compare each tracked file's blob hash with both the index and the working tree. If a file is modified in the working directory and also differs in the target branch, checkout must refuse to avoid overwriting uncommitted changes.

Q5.3: "Detached HEAD" means HEAD contains a commit hash directly instead of a branch reference. What happens if you make commits in this state? How could a user recover those commits?

Ans: In detached HEAD, new commits are created normally, but they are not attached to any branch and can become unreachable later. To recover them, the user can use the commit hash directly, or find it in the reflog and create a branch from it.

Garbage Collection and Space Reclamation

Q6.1: Over time, the object store accumulates unreachable objects — blobs, trees, or commits that no branch points to (directly or transitively). Describe an algorithm to find and delete these objects. What data structure would you use to track "reachable" hashes efficiently? For a repository with 100,000 commits and 50 branches, estimate how many objects you'd need to visit.

Ans: Start from all branch tips, walk commit parents backward, and mark every reachable commit, tree, and blob using a hash set or visited map. For 100,000 commits and 50 branches, you would visit roughly the reachable commit graph once, so on the order of the commit count plus reachable trees/blobs, not all stored objects.

Q6.2: Why is it dangerous to run garbage collection concurrently with a commit operation? Describe a race condition where GC could delete an object that a concurrent commit is about to reference. How does Git's real GC avoid this?

Ans: It is dangerous because GC may mark objects as unreachable before a concurrent commit finishes writing refs or files that point to them. A race can delete an object that the new commit is about to use; real Git avoids this with locks, reflogs, grace periods, and by only pruning objects that have been unreachable for long enough.